

k8s-05

1. Create a namespace dev-environment and apply a resource-based quota that restricts the number of pods to 3 and services to 2.

- Create the namespace and Verify:

```
root@Master:~# kubectl create namespace dev-environment
namespace/dev-environment created
root@Master:~# kubectl get ns
NAME                STATUS    AGE
default             Active    6d
dev-environment     Active    8s
kube-node-lease     Active    6d
kube-public         Active    6d
kube-system         Active    6d
```

- Create resource quota. yaml

```
root@Master:~# mkdir -p ~/k8s-05
root@Master:~# cd ~/k8s-05
root@Master:~/k8s-05# vi resourcequota.yaml
```

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-resourcequota
  namespace: dev-environment
spec:
  hard:
    pods: "3"
    services: "2"
```

- Apply the Resource Quota and verify

```
root@Master:~/k8s-05# kubectl apply -f resourcequota.yaml
resourcequota/dev-resourcequota created
root@Master:~/k8s-05# kubectl get resourcequota -n dev-environment
NAME                AGE    REQUEST                LIMIT
dev-resourcequota   11s    pods: 0/3, services: 0/2
```

```
root@Master:~/k8s-05# kubectl run pod1 --image=nginx -n dev-environment
pod/pod1 created
root@Master:~/k8s-05# kubectl run pod2 --image=nginx -n dev-environment
pod/pod2 created
root@Master:~/k8s-05# kubectl run pod3 --image=nginx -n dev-environment
pod/pod3 created
root@Master:~/k8s-05# kubectl run pod4 --image=nginx -n dev-environment
Error from server (Forbidden): pods "pod4" is forbidden: exceeded quota: dev-resourcequota, requested:
```

```
root@Master:~/k8s-05# kubectl describe resourcequota dev-resourcequota -n dev-environment
Name:                dev-resourcequota
Namespace:           dev-environment
Resource              Used  Hard
-----
pods                  3    3
services              0    2
```

2. Create a pod in the prod-environment namespace with 0.2 CPU and 200Mi memory requests, and 0.5 CPU and 500Mi memory limits.

- Create the namespace and verify

```
root@Master:~/k8s-05# kubectl create namespace prod-environment
namespace/prod-environment created
```

```
root@Master:~/k8s-05# kubectl get ns
NAME                STATUS    AGE
default              Active    6d
dev-environment      Active    9m40s
kube-node-lease      Active    6d
kube-public          Active    6d
kube-system          Active    6d
prod-environment     Active    18s
```

- Create a yaml file

```
root@Master:~/k8s-05# vi prod-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: prod-pod
  namespace: prod-environment
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        cpu: "200m"
        memory: "200Mi"
      limits:
        cpu: "500m"
        memory: "500Mi"
```

- Apply and verify it

```
root@Master:~/k8s-05# kubectl apply -f prod-pod.yaml
pod/prod-pod created
root@Master:~/k8s-05# kubectl get pod -n prod-environment
NAME        READY   STATUS    RESTARTS   AGE
prod-pod    1/1     Running   0           10s
```

- Check Limits

```
root@Master:~/k8s-05# kubectl describe pod prod-pod -n prod-environment | grep -A10 "Limits"
Limits:
  cpu: 500m
  memory: 500Mi
Requests:
  cpu: 200m
  memory: 200Mi
Environment: <none>
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-xdwnd (ro)
Conditions:
  Type              Status
```

3. In the staging-environment namespace, set a Limit Range that assigns default CPU and memory limits (300m CPU, 600Mi memory) and applies a minimum and maximum CPU.

- Create the namespace and create yaml file

```
root@Master:~/k8s-05# kubectl create namespace staging-environment
namespace/staging-environment created
root@Master:~/k8s-05# vi limitrange.yaml
root@Master:~/k8s-05#
```

```
apiVersion: v1
kind: LimitRange
metadata:
  name: staging-limitrange
  namespace: staging-environment
spec:
  limits:
  - type: Container
    default:
      cpu: "300m"
      memory: "600Mi"
    defaultRequest:
      cpu: "200m"
      memory: "300Mi"
    min:
      cpu: "100m"
    max:
      cpu: "1000m"
```

- Apply the Limit Range

```
root@Master:~/k8s-05# kubectl apply -f limitrange.yaml
limitrange/staging-limitrange created
root@Master:~/k8s-05# kubectl get limitrange -n staging-environment
```

NAME	CREATED AT
staging-limitrange	2025-07-25 14:54:00

- Verify the Limit Range

```
root@Master:~/k8s-05# kubectl describe limitrange staging-limitrange -n staging-environment
```

Name:	staging-limitrange					
Namespace:	staging-environment					
Type	Resource	Min	Max	Default Request	Default Limit	Max Limit/Request Ratio
Container	cpu	100m	1	200m	300m	-
Container	memory	-	-	300Mi	600Mi	-

- Test the default values

```
root@Master:~/k8s-05# kubectl run test-limit-pod \
  --image=nginx \
  -n staging-environment
pod/test-limit-pod created
```

```
root@Master:~/k8s-05# kubectl describe pod test-limit-pod -n staging-environment | grep -A10 "Limits"
```

```
Limits:
  cpu:    300m
  memory: 600Mi
Requests:
  cpu:    200m
  memory: 300Mi
Environment:  <none>
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-x7sdt (ro)
Conditions:
  Type              Status
root@Master:~/k8s-05#
```

4. Create a pod and a Node Port service in the default namespace, then create another pod in the test namespace and communicate between them using Service DNS.

- Create a Pod in the default namespace

```
root@Master:~/k8s-05# vi default-pod.yaml
root@Master:~/k8s-05#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-default
  labels:
    app: nginx-default
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

- Apply it

```
root@Master:~/k8s-05#
root@Master:~/k8s-05# kubectl apply -f default-pod.yaml
pod/nginx-default unchanged
```

- Create the Service YAML and written yaml file

```
root@Master:~/k8s-05# vi default-service.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx-default
  type: NodePort
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

- Apply the service and verify the endpoints

```
root@Master:~/k8s-05# kubectl apply -f default-service.yaml
service/nginx-service created
root@Master:~/k8s-05# kubectl get svc nginx-service
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
nginx-service       NodePort    10.104.0.105   <none>         80:30082/TCP     10s
root@Master:~/k8s-05# kubectl get endpoints nginx-service
NAME                ENDPOINTS          AGE
nginx-service       192.168.219.82:80  23s
```

- Create the test namespace and Create the Busy Box Pod

```
root@Master:~/k8s-05# kubectl create namespace test
namespace/test created
root@Master:~/k8s-05# kubectl run dns-test \
  --image=busybox \
  --restart=Never \
  -n test \
  -- sleep 3600
pod/dns-test created
```

- Check status

```
root@Master:~/k8s-05# kubectl get pods -n test
NAME          READY   STATUS    RESTARTS   AGE
dns-test      1/1     Running   0           9s
root@Master:~/k8s-05# kubectl exec -it dns-test -n test
```

- Test Service DNS

```
root@Master:~/k8s-05# kubectl exec -it dns-test -n test -- nslookup nginx
Server:          10.96.0.10
Address:         10.96.0.10:53

Name:   nginx-service.default.svc.cluster.local
Address: 10.104.0.105
```

5. Apply a Limit Range with a max limit/request ratio of 2 for memory in the performance-environment namespace, and test by creating a pod with mismatched resource requests and limits.

- Create the namespace

```
root@Master:~/k8s-05# kubectl create namespace performance-environment
namespace/performance-environment created
root@Master:~/k8s-05#
```

- Create ratio-limit range. Yaml

```
namespace/performance-environment created
root@Master:~/k8s-05# vi ratio-limitrange.yaml
root@Master:~/k8s-05#
```

```
apiVersion: v1
kind: LimitRange
metadata:
  name: performance-limitrange
  namespace: performance-environment
spec:
  limits:
  - type: Container
    maxLimitRequestRatio:
      memory: "2"
    defaultRequest:
      memory: 200Mi
    default:
      memory: 400Mi
```

- Apply the Limit Range and verify it

```
root@Master:~/k8s-05# kubectl apply -f ratio-limitrange.yaml
limitrange/performance-limitrange created
root@Master:~/k8s-05# kubectl describe limitrange performance-limitrange -n performance-environment
Name:         performance-limitrange
Namespace:    performance-environment
Type          Resource  Min  Max  Default Request  Default Limit  Max Limit/Request Ratio
-----
Container     memory    -    -    200Mi            400Mi          2
root@Master:~/k8s-05#
```

- Create a Pod that Violates the Ratio

```
Container     memory    -    -    200Mi
root@Master:~/k8s-05# vi bad-pod.yaml
root@Master:~/k8s-05#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: bad-pod
  namespace: performance-environment
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "200Mi"
      limits:
        memory: "600Mi"
```

- Apply it

```
root@Master:~/k8s-05# kubectl apply -f bad-pod.yaml
Error from server (Forbidden): error when creating "bad-pod.yaml": pods "bad-pod" is
is 3.000000
root@Master:~/k8s-05#
```

- Create a Valid Pod

```
root@Master:~/k8s-05# vi good-pod.yaml
root@Master:~/k8s-05# kubectl apply -f good-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: good-pod
  namespace: performance-environment
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "200Mi"
      limits:
        memory: "400Mi"
```

- Apply and verify it

```
root@Master:~/k8s-05# kubectl apply -f good-pod.yaml
pod/good-pod created
root@Master:~/k8s-05# kubectl get pods -n performance-environment
NAME        READY   STATUS    RESTARTS   AGE
good-pod    1/1     Running   0           6s
root@Master:~/k8s-05#
```